

```
return(triangle)
```

---

It will be helpful to be able to visualize the triangles we're working with. Let's create a simple function that will take any triangle and plot it. First, we'll use the `genlines()` function that we defined in Chapter 6. Remember that this function takes a collection of points and converts them into lines. Again, it's a very simple function, just appending points to a list called `lines`:

---

```
def genlines(listpoints, itinerary):
    lines = []
    for j in range(len(itinerary)-1):
        lines.append([listpoints[itinerary[j]], listpoints[itinerary[j+1]]])
    return(lines)
```

---

Next, we'll create our simple plotting function. It will take a triangle we pass to it, split it into its *x* and *y* values, call `genlines()` to create a collection of lines based on those values, plot the points and lines, and finally save the figure to a *.png* file. It uses the `pylab` module for plotting and code from the `matplotlib` module to create the line collection. [Listing 7-1](#) shows this function.

---

```
import pylab as pl
from matplotlib import collections as mc
def plot_triangle_simple(triangle, thename):
    fig, ax = pl.subplots()

    xs = [triangle[0][0], triangle[1][0], triangle[2][0]]
    ys = [triangle[0][1], triangle[1][1], triangle[2][1]]

    itin=[0,1,2,0]

    thelines = genlines(triangle, itin)

    lc = mc.LineCollection(genlines(triangle, itin), linewidths=2)

    ax.add_collection(lc)

    ax.margins(0.1)
    pl.scatter(xs, ys)
    pl.savefig(str(thename) + '.png')
    pl.close()
```

---